Laporan Tugas Besar 1

IF2211 Strategi Algoritma

Battle Code 2

Semester II Tahun 2025/2026

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Deskripsi Tugas **2**
1.1 Latar Belakang . . . 2
1.2 Komponen Permainan . . . 2
1.2.1 Objective Permainan . . . 2
1.2.2 Aturan Tiebreaker . . . 2
1.2.3 Peta Permainan . . . 2
1.2.4 Jarak Pandang dan Pergerakan . . . 2
1.2.5 Tower dan Ruins . . . 3
1.2.6 Sumber Daya . . . 3
1.2.7 Robot . . . 3
1.2.8 Jenis Robot . . . 3
1.2.9 Komunikasi . . . 4
1.2.10 Batasan Program . . . 4
1.2.11 Tujuan Tugas Besar . . . 4

2 Landasan Teori **4**
2.1 Algoritma Greedy . . . 4
2.1.1 Konsep Dasar . . . 4
2.1.2 Elemen-Elemen . . . 5
2.2 Mekanisme Kerja Program dan Implementasi Greedy pada Battlecode . . . 5
2.2.1 Sistem Eksekusi Berbasis Giliran (Turn-Based Execution) . . . 5
2.2.2 Penerapan Algoritma Greedy pada Bot . . . 6
2.2.3 Eksekusi dan Siklus Hidup Bot . . . 6

3 Aplikasi Strategi *Greedy* **6**
3.1 Pemetaan Persoalan Battlecode 2025 ke Elemen-Elemen Algoritma *Greedy* . . . 6
3.1.1 Himpunan Kandidat . . . 6
3.1.2 Himpunan Solusi . . . 6
3.1.3 Fungsi Seleksi . . . 6
3.1.4 Fungsi Kelayakan . . . 6
3.1.5 Fungsi Solusi . . . 6
3.1.6 Fungsi Objektif . . . 7
3.2 Eksplorasi Tiga Alternatif Solusi *Greedy* . . . 7
3.2.1 Alternatif 1 — *Greedy* Ekspansi Teritorial Berbasis Nilai Tile Lokal . . . 7
3.2.2 Alternatif 2 — *(Dikembangkan oleh anggota kelompok lain)* . . . 8
3.2.3 Alternatif 3 — *(Dikembangkan oleh anggota kelompok lain)* . . . 9
3.3 Analisis Efisiensi dan Efektivitas . . . 9
3.4 Strategi *Greedy* yang Dipilih . . . 9

1 Deskripsi Tugas

1.1 Latar Belakang

Battlecode merupakan permainan strategi waktu nyata di mana pemain menulis kode untuk mengendalikan agen otonom berupa robot. Robot-robot tersebut harus bekerja sama untuk mengalahkan tim lawan dengan mengelola sumber daya, melakukan koordinasi, serta menerapkan strategi yang efektif.

Dalam permainan ini, pemain harus memanfaatkan berbagai konsep ilmu komputer seperti kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, serta mekanisme komunikasi antar agen. Selain itu, setiap robot memiliki batasan sumber daya komputasi pada setiap giliran, sehingga pemain perlu merancang algoritma yang efisien dan mampu mengambil keputusan dengan cepat.

Pada Tugas Besar 1 mata kuliah IF2211 Strategi Algoritma ini, mahasiswa diminta untuk merancang dan mengimplementasikan bot permainan Battlecode 2025 yang memanfaatkan strategi algoritma greedy. Bot yang dibuat nantinya akan dipertandingkan dengan bot dari kelompok lain.

1.2 Komponen Permainan

1.2.1 Objective Permainan

Tujuan utama permainan Battlecode adalah mewarnai wilayah peta sebanyak mungkin. Tim yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai akan memenangkan permainan.

Selain itu, kemenangan juga dapat dicapai dengan menghancurkan seluruh robot dan menara milik tim lawan.

1.2.2 Aturan Tiebreaker

Apabila setelah 2000 ronde tidak ada tim yang berhasil mewarnai lebih dari 70% peta, maka permainan akan berakhir dan pemenang ditentukan berdasarkan kriteria berikut secara berurutan:

1. Luas area peta yang telah diwarnai
2. Jumlah menara sekutu yang masih hidup
3. Total jumlah uang (chips)
4. Total jumlah cat (paint) yang dimiliki robot dan menara
5. Jumlah robot sekutu yang masih hidup
6. Jika masih seri, pemenang ditentukan secara acak

1.2.3 Peta Permainan

Setiap permainan dimainkan pada peta berbentuk grid dua dimensi berbentuk persegi panjang dengan ukuran antara 20×20 hingga 60×60 petak.

Koordinat peta dimulai dari titik kiri bawah $(0, 0)$ dan meningkat ke arah timur (kanan) dan utara (atas). Setiap lokasi pada peta direpresentasikan oleh objek `MapLocation` yang menyimpan koordinat x dan y.

Untuk menjaga keseimbangan permainan, peta dijamin memiliki sifat simetris, baik melalui rotasi maupun refleksi.

1.2.4 Jarak Pandang dan Pergerakan

Robot dapat melihat lingkungan di sekitarnya dalam radius $\sqrt{20}$ atau sekitar 4.47 petak. Robot tidak dapat bergerak melewati menara maupun robot lain.

Peta juga dapat memiliki tembok (walls) yang tidak dapat dilewati maupun diwarnai. Jumlah tembok maksimal adalah 20% dari seluruh petak pada peta. Selain itu, robot juga tidak dapat melewati atau mewarnai reruntuhan (ruins).

1.2.5 Tower dan Ruins

Menara memiliki fungsi sebagai zona spawn tim sekaligus unit pertahanan yang dapat menyerang musuh. Untuk membangun menara, pemain harus mewarnai pola khusus pada area reruntuhan (ruins) berukuran 5×5.

Setiap permainan dimulai dengan sejumlah reruntuhan yang telah tersedia pada peta. Reruntuhan tidak dapat dipindahkan maupun dihancurkan. Pusat reruntuhan memiliki jarak minimal lima petak satu sama lain, dan area di sekitarnya dijamin bebas dari tembok.

Setiap tim memulai permainan dengan dua menara, yaitu:

- Menara penghasil uang (Money Tower)
- Menara penghasil cat (Paint Tower)

1.2.6 Sumber Daya

Dalam permainan Battlecode terdapat dua sumber daya utama, yaitu **paint** dan **chips**.

- Paint

  Paint merupakan sumber daya utama yang digunakan untuk mewarnai wilayah peta dan menentukan skor tim. Setiap robot dan menara memiliki kapasitas penyimpanan paint yang berbeda.Robot yang memiliki paint rendah akan mengalami peningkatan cooldown aksi, bahkan tidak dapat melakukan aksi apabila paint habis sepenuhnya.

- Chips

  Chips digunakan untuk membangun robot maupun menara. Chips dihasilkan secara pasif oleh menara penghasil uang setiap ronde.Setiap tim memulai permainan dengan 2500 chips.

1.2.7 Robot

Robot merupakan unit utama dalam permainan. Setiap robot dapat melakukan berbagai aksi seperti bergerak, menyerang, melakukan sensing, serta berkomunikasi dengan robot lain.

Setiap robot memiliki batas komputasi yang disebut *bytecode*. Jika robot melebihi batas bytecode pada satu giliran, eksekusi program akan dihentikan sementara dan dilanjutkan pada ronde berikutnya.

Robot memiliki atribut utama berupa:

- Hitpoints (HP)
- Kapasitas paint
- Cooldown aksi dan pergerakan
- Kemampuan khusus sesuai tipe robot

1.2.8 Jenis Robot

Permainan Battlecode menyediakan beberapa jenis robot dengan karakteristik berbeda.

- Soldier

  Robot tipe Soldier memiliki kemampuan menyerang satu petak dan dapat memberikan damage terhadap menara musuh.

- Splasher

  Robot tipe Splasher memiliki serangan area (area of effect) yang mampu mewarnai atau menimpa cat pada beberapa petak sekaligus.

- Mopper

  Robot tipe Mopper memiliki kemampuan untuk menghapus cat milik musuh serta mentransfer paint kepada robot atau menara sekutu.

1.2.9 Komunikasi

Robot tidak dapat berbagi variabel secara langsung karena setiap robot menjalankan salinan program yang terpisah. Oleh karena itu, komunikasi dilakukan melalui dua mekanisme utama:

- **Pesan (messages)** yang dapat dikirim antar robot atau menara
- **Penanda (markers)** yang ditempatkan pada peta untuk memberikan informasi kepada robot sekutu

Setiap pesan memiliki ukuran maksimal 4 byte dan hanya disimpan selama lima ronde dalam buffer penerima.

1.2.10 Batasan Program

Program robot memiliki beberapa batasan yang harus dipatuhi.

- Bytecode

  Setiap robot memiliki batas komputasi sebesar 17500 bytecode per ronde, sedangkan menara memiliki batas 20000 bytecode per ronde.

- Memori

  Penggunaan memori robot tidak boleh melebihi 8 MB selama pertandingan.

- Execution Time

  Jika total waktu eksekusi sebuah tim melebihi batas yang ditentukan oleh sistem, maka tim tersebut akan otomatis kalah.

1.2.11 Tujuan Tugas Besar

Pada tugas besar ini, setiap kelompok diminta untuk:

- Mengembangkan tiga bot Battlecode menggunakan bahasa Java
- Mengimplementasikan strategi algoritma greedy pada masing-masing bot
- Menggunakan heuristic yang berbeda untuk setiap strategi greedy
- Mengoptimalkan strategi agar mampu memenangkan pertandingan

Bot yang dikembangkan akan dipertandingkan dengan bot dari kelompok lain dalam kompetisi yang diselenggarakan pada akhir tugas besar.

2 Landasan Teori

2.1 Algoritma Greedy

2.1.1 Konsep Dasar

Algoritma *greedy* merupakan suatu metode sederhana yang paling populer untuk memecahkan persoalan optimasi (*optimization problems*), yaitu persoalan mencari solusi optimal. Terdapat dua jenis persoalan optimasi:

- Maksimasi (*maximization*) - contohnya adalah profit, reward, dan lain sebagainya.
- Minimasi (*minimization*) - contohnya adalah *cost*, waktu, jarak, dan lain sebagainya.

Algoritma ini menyarankan pembentukan sebuah solusi secara bertahap, di mana setiap langkah memiliki banyak pilihan yang bisa dievaluasi hingga akhirnya solusi lengkap untuk masalah tersebut bisa tercapai. Penentuan solusi yang bisa dievaluasi pada setiap tahap harus memenuhi kriteria-kriteria berikut:

- Pilihan tidak boleh melanggar *constraints* dari permasalah yang sedang diselesaikan
- Pilihan yang diambil pada tiap langkahnya haruslah merupakan pilihan yang optimal secara lokal (pilihan terbaik yang tersedia pada tahap tersebut)

- Pilihan yang sudah dibuat bersifat final dan tidak dapat diubah maupun ditarik kembali pada langkah-langkah berikutnya dalam algoritma.

Tiga karakteristik barusan menjelaskan mengapa algoritma ini disebut sebagai algoritma *greedy*. Setiap langkah yang diambil harus memenuhi prinsip "*take what you can get now!*" tanpa perlu memikirkan akibat yang dihasilkan di langkah-langkah berikutnya. Setiap langkah optimum lokal yang diambil diharapkan bisa menghasilkan solusi keseluruhan yang optimum secara global.

Algoritma ini secara spesifik hanya bisa diaplikasikan untuk tipe permasalahan optimasi. Tipe persoalan optimasi adalah persoalan yang memiliki tujuan untuk mencari solusi optimal dari berbagai kemungkinan solusi yang ada. Secara umum, permalasahan optimasi dibagi menjadi dua jenis, maksimasi (*maximization*), yaitu persoalan optimasi yang bertujuan untuk mendapatkan hasil maksimal dari fungsi tujuan, dan minimasi (*minimization*), yaitu persoalan optimasi yang bertujuan untuk mendapatkan hasil minimal dari fungsi tujuan.

Walaupun setiap langkah yang diambil merupakan optimum lokal dari semua pilihan yang tersedia, perlu diingat bahwa solusi yang dihasilkan belum tentu merupakan sebuah optimum global. Pendekatan menggunakan algoritma ini bisa dijadikan sebagai solusi hampiran, yang pada beberapa jenis persoalan tertentu sudah dianggap cukup dan dapat diterima, mengingat efisiensi waktu yang jauh lebih cepat dibanding dengan algoritma *exhaustive search*.

2.1.2 Elemen-Elemen

Dalam memecahkan tipe permasalahan optimasi, algoritma *greedy* melibatkan pencarian sebuah himpunan bagian dari sekumpulan kandidat yang ada. Himpunan bagian tersebut harus memenuhi beberapa kriteria tertentu agar bisa merepresentasikan suatu solusi yang dioptimalkan oleh sebuah fungsi tujuan. Untuk mencapai hal terrsebut, algoritma *greedy* biasanya dibangun oleh beberapa elemen pendukung:

1. **Himpunan Kandidat (*Candidate set*)**
   Ini adalah sekumpulan elemen ytang berisi seluruh kandidat pilihan yang bisa dipilih pada setiap langkah pencarian. Sebagai contoh, dalam kasus komputasi, kandidat dapat beruapa simpul ataupun sisi di dalam graf, pecahan mata uang, kumpulan benda berbobot tertentu, dan hal-hal lain yang relevan dengan permasalahan yang sedang dioptimasi.

2. **Himpunan Solusi (*Solution set*)**
   Himpunan solusi adalah sebuah himpunan yang berisi solusi-solusi yang dipilih pada setiap langkah yang sudah diambil. Himpunan ini dibangun secara bertahap dengan kondisi awal kosong dan akan terus ditambahkan elemen baru berupa solusi yang dipilih pada setiap iterasi pemilihan langkah hingga akhir solusi akhir tercapai.

3. **Fungsi Seleksi (*Selection function*)**
   Fungsi ini bertugas untuk memiliki kandidat terbaik dari himpunan kandidat pada suatu langkah. Fungsi ini didasarkan pada strategi *greedy* (heuristik) tertentu yang memberikan kondisi lokal optimum pada langkah tersebut.

4. **Fungsi Kelayakan (*Feasible function*)**
   Fungsi ini bertugas untuk menguji output fungsi seleksi untuk menentukan layak atau tidaknya hasil tersebut dimasukkan ke himpunan solusi. Pengcekan ini penting untuk memastikan bahwa output dari fungsi seleksi tidak melanggar *constraint* yang ditetapkan oleh permasalah.

5. **Fungsi Objektif (*Objective function*)**
   Fungsi ini bertugas untuk mengukur ketercapaian keseluruhan ataupun nilai dari sebuah solusi yang dihasilkan. Inilah fungsi utama yang bertugas untuk mencapai target akhir persoalan, yaitu memaksimalisasi atau meminimalisasi nilai dari solusi yang didapatkan.

2.2 Mekanisme Kerja Program dan Implementasi Greedy pada Battlecode

2.2.1 Sistem Eksekusi Berbasis Giliran (Turn-Based Execution)

Dalam permainan Battlecode, setiap robot menjalankan siklus hidupnya melalui struktur *infinite loop* yang mengeksekusi instruksi pada setiap ronde (*turn*). Karena adanya limitasi *bytecode* (17.500 untuk robot, 20.000 untuk menara), robot tidak dapat melakukan komputasi yang berat seperti algoritma penelusuran jalur (*pathfinding*) global secara menyeluruh pada setiap giliran. Oleh karena itu, program menggunakan pola *Finite State Machine* (FSM) di mana robot akan mengevaluasi kondisi sekitarnya dan memutuskan transisi *state* (misalnya: `EXPLORE`, `PAINT`, `BUILD_TOWER`).

2.2.2 Penerapan Algoritma Greedy pada Bot

Algoritma *greedy* diimplementasikan sebagai inti dari pengambilan keputusan robot, baik untuk pergerakan (*movement*) maupun aksi (*action*). Implementasi ini dilakukan melalui pendekatan heuristik spasial berbobot (*weighted spatial heuristic*).

Alih-alih merencanakan serangkaian langkah panjang yang menguras *bytecode*, robot akan mengevaluasi petak-petak (*tiles*) yang berada tepat di sekitarnya. Robot memberikan skor pada setiap petak berdasarkan parameter taktis (seperti keberadaan cat musuh, reruntuhan, atau teman yang berdempetan). Keputusan *greedy* diambil dengan memilih petak yang memberikan skor heuristik tertinggi (optimum lokal) pada giliran tersebut.

2.2.3 Eksekusi dan Siklus Hidup Bot

Secara arsitektur, program dijalankan melalui sebuah kelas utama `RobotPlayer` yang bertindak sebagai titik masuk (*entry point*). Berdasarkan `UnitType`, kontrol dilempar ke kelas spesifik pewaris `BaseRobot` (seperti `SoldierUnit`, `SplasherUnit`, dll.). Setiap giliran, agen robot akan memanggil fungsi `tick()`, membaca pesan dari *broadcasting* menara, menjalankan fungsi *anti-clustering* (penghindar dempet), dan mengeksekusi aksi *greedy*-nya sebelum akhirnya memanggil `Clock.yield()` untuk menyerahkan sisa eksekusi kepada *engine* permainan.

3 Aplikasi Strategi *Greedy*

3.1 Pemetaan Persoalan Battlecode 2025 ke Elemen-Elemen Algoritma *Greedy*

Permainan Battlecode 2025 dapat dimodelkan sebagai sebuah persoalan optimasi yang diselesaikan secara inkremental, di mana setiap robot membuat keputusan aksi terbaik pada setiap ronde berdasarkan informasi lokal yang tersedia. Pemetaan antara elemen-elemen persoalan Battlecode dan elemen-elemen algoritma *greedy* [**?**] adalah sebagai berikut.

3.1.1 Himpunan Kandidat

Himpunan kandidat C terdiri atas seluruh aksi yang secara teknis dapat dilaksanakan oleh sebuah robot pada satu ronde. Aksi-aksi tersebut meliputi pilihan arah gerak ke delapan tile tetangga, pilihan tile yang dapat diserang atau diwarnai dalam radius aksi, serta aksi-aksi khusus sesuai kemampuan tipe robot masing-masing.

3.1.2 Himpunan Solusi

Himpunan solusi S adalah rangkaian aksi yang telah dipilih dan dilaksanakan oleh sebuah robot sepanjang permainan. Himpunan ini dibangun secara bertahap: setiap ronde, satu aksi baru ditambahkan ke dalam S berdasarkan hasil evaluasi fungsi seleksi.

3.1.3 Fungsi Seleksi

Fungsi seleksi menentukan aksi terbaik dari himpunan kandidat C pada setiap langkah. Fungsi ini diwujudkan melalui heuristik penilaian yang disesuaikan dengan tujuan masing-masing tipe robot: robot penyerang memilih tile yang paling menguntungkan untuk diwarnai, robot pembersih memilih cat musuh yang paling mengancam area sekutu, dan menara memilih jenis unit yang paling dibutuhkan berdasarkan komposisi pasukan saat ini.

3.1.4 Fungsi Kelayakan

Fungsi kelayakan memastikan bahwa aksi yang dipilih oleh fungsi seleksi dapat benar-benar dilaksanakan. Setiap kandidat aksi diperiksa kelayakannya terlebih dahulu sebelum dieksekusi, misalnya memastikan tile tujuan dapat dijangkau, bahwa *cooldown* robot sudah siap, serta bahwa sumber daya yang diperlukan tersedia.

3.1.5 Fungsi Solusi

Fungsi solusi dikelola sepenuhnya oleh *game engine*. Kondisi solusi terpenuhi apabila salah satu kondisi kemenangan tercapai: persentase tile bercat tim melebihi 70%, atau seluruh robot dan menara lawan berhasil dihancurkan.

3.1.6 Fungsi Objektif

Fungsi objektif utama yang ingin dimaksimalkan adalah persentase tile yang berhasil diwarnai oleh tim:

$$f_{\text{obj}} = \frac{|\text{tile bercat tim sendiri}|}{|\text{total tile yang dapat diwarnai}|} \times 100\% \tag{1}$$

Kemenangan diraih apabila $f_{\text{obj}} > 70\%$. Secara lokal, setiap robot berusaha memaksimalkan kontribusinya terhadap (1) dengan selalu memilih aksi yang memberikan peningkatan terbesar terhadap cakupan wilayah tim pada ronde yang sedang berjalan.

3.2 Eksplorasi Tiga Alternatif Solusi *Greedy*

Tiga alternatif strategi *greedy* dikembangkan dengan heuristik yang berbeda satu sama lain. Perbedaan ini diwujudkan terutama pada fungsi seleksi, yang merupakan elemen paling krusial dalam menentukan perilaku algoritma *greedy* [**?**]. Setiap subbagian berikut terlebih dahulu menguraikan tujuan dan heuristik umum setiap alternatif, kemudian mendeskripsikan pemetaan elemen-elemen *greedy* secara detail untuk alternatif tersebut.

3.2.1 Alternatif 1 — *Greedy* Ekspansi Teritorial Berbasis Nilai Tile Lokal

Tujuan dan Heuristik Umum

Strategi ini mengadopsi prinsip: setiap robot pada setiap ronde memilih aksi yang memberikan nilai heuristik tertinggi berdasarkan kondisi tile yang dapat diinderanya saat itu, tanpa memperhitungkan konsekuensi jangka panjang. Nilai tersebut ditentukan oleh jenis cat pada tile tujuan, jarak ke reruntuhan terdekat, serta riwayat kunjungan robot untuk mencegah osilasi. Robot beroperasi penuh hingga cat habis; tidak terdapat mekanisme pengisian ulang cat (*refill*). Setelah cat habis, robot menjauh dari markas agar tidak menghalangi robot-robot baru.

Himpunan Kandidat

Himpunan kandidat C pada setiap ronde terdiri atas:

- delapan arah gerak beserta pilihan diam;
- seluruh tile non-sekutu dalam radius aksi yang dapat diserang atau diwarnai; serta
- reruntuhan dalam radius sensor yang dapat diinisiasi pola towernya.

Himpunan Solusi

Himpunan solusi S adalah urutan pasangan aksi (gerak, serang/warnai) yang dilaksanakan robot sepanjang permainan. Setiap ronde menyumbangkan tepat satu elemen ke dalam S.

Fungsi Seleksi

Fungsi seleksi berbeda untuk setiap tipe robot.

Untuk *robot penyerang* (*Soldier*), aksi dipilih melalui sebuah *state machine* berprioritas:

1. Pada fase awal permainan, dua *Soldier* pertama mengecat area sekitar menara awal secara simetris berdasarkan paritas ID robot, sehingga keduanya tidak tumpang-tindih.
2. Jika terdapat reruntuhan tanpa tower dalam radius sensor, *Soldier* memprioritaskan penyelesaian pola tower. Reruntuhan dipilih dengan skor:

$$\text{ruinScore}(r) = \text{correctTiles}(r) \times 10 - \frac{d^2_{\text{robot},r}}{5} \tag{2}$$

   di mana $\text{correctTiles}(r)$ adalah jumlah tile di area 5×5 sekitar reruntuhan r yang sudah diwarnai sesuai pola, dan $d^2_{\text{robot},r}$ adalah jarak kuadrat robot ke r.

3. Jika terdapat tile non-sekutu dalam radius aksi, *Soldier* memilih arah gerak dengan skor:

$$\text{moveScore}(l) = v(l) + b_{\text{ruin}}(l) - \text{trailPenalty}(l) \tag{3}$$

   dengan nilai cat:

$$v(l) = \begin{cases} +60 & \text{jika tile } l \text{ bercat musuh,} \\ +30 & \text{jika tile } l \text{ belum diwarnai,} \\ -15 & \text{jika tile } l \text{ bercat sekutu;} \end{cases} \tag{4}$$

   bonus reruntuhan $b_{\text{ruin}}(l) = +80$ jika ada reruntuhan dalam radius 8 dari l; dan penalti anti-osilasi $\text{trailPenalty}(l) = 800 \times |\{t \in \text{trail}[50] : t = l\}|$.

4. Jika tidak ada target pada kondisi di atas, robot menjelajah ke titik kandidat terjauh dari posisi saat ini. Jika cat mendekati nol, robot menjauh dari markas.

Untuk *robot serangan area* (*Splasher*), terdapat dua lapisan seleksi. Pertama, pemilihan titik serangan berdasarkan:

$$\text{splashScore}(l) = \sum_{t \in \text{area}(l,\, r=\sqrt{4})} \big(3 \cdot \mathbf{1}_{\text{enemy}}(t) + 1 \cdot \mathbf{1}_{\text{empty}}(t)\big) \tag{5}$$

dengan ambang minimum $\text{splashScore}(l) \geq 3$. Kedua, pemilihan arah gerak berdasarkan *density* cat non-sekutu per kuadran:

$$\text{density}(q) = \frac{\text{quadrantScore}[q]}{\text{quadrantCount}[q]}, \quad q \in \{\text{NE, NW, SE, SW}\} \tag{6}$$

Robot bergerak ke kuadran dengan $\text{density}(q)$ tertinggi, dengan syarat $\text{density}(q) > 0{,}3$. Jika cat habis, robot menjauh dari markas.

Untuk *robot pembersih* (*Mopper*), state dievaluasi berurutan: pertarungan jarak dekat → pembersihan cat → dukungan sekutu → patroli. Pada pertarungan jarak dekat, dipilih arah ayunan yang memaksimalkan musuh yang terkena:

$$d^* = \arg\max_{d \in \text{DIRS}} \text{swingHits}(d) \tag{7}$$

Pada pembersihan cat, target dipilih berdasarkan:

$$\text{cleanScore}(l) = \max\big(0,\ 100 - d^2_{\text{base}}(l)\big) + 5 \cdot n_{\text{adj}}(l) \tag{8}$$

di mana $d^2_{\text{base}}(l)$ adalah jarak kuadrat ke menara sekutu terdekat dan $n_{\text{adj}}(l)$ adalah jumlah tile bercat musuh yang bersebelahan dengan l.

Untuk *menara* (*Tower*), jenis unit yang di-*spawn* dipilih secara bertingkat: pada ronde awal (< 150) bergantian antara penyerang dan serangan area; setelahnya, jenis unit dipilih berdasarkan cek rasio berurutan dengan target 15% pembersih, 40% penyerang, dan 45% serangan area.

Fungsi Kelayakan

Setiap kandidat aksi diperiksa melalui fungsi-fungsi bawaan *game engine* sebelum dieksekusi: apakah tile tujuan dapat dicapai, apakah *cooldown* sudah siap, dan apakah tile target berada dalam radius aksi yang valid.

Fungsi Solusi

Kondisi solusi tercapai ketika $f_{\text{obj}} > 70\%$ atau seluruh unit musuh hancur, sebagaimana didefinisikan oleh *game engine*.

Fungsi Objektif

Secara lokal, setiap robot memaksimalkan jumlah tile baru yang berhasil diwarnai pada setiap ronde, yang secara kumulatif berkontribusi pada f_{obj} (1).

3.2.2 Alternatif 2 — *(Dikembangkan oleh anggota kelompok lain)*

Tujuan dan Heuristik Umum

(Diisi oleh anggota kelompok yang mengembangkan Alternatif 2.)

Himpunan Kandidat

(Diisi oleh anggota kelompok yang mengembangkan Alternatif 2.)

Himpunan Solusi

(Diisi oleh anggota kelompok yang mengembangkan Alternatif 2.)

Fungsi Seleksi

(Diisi oleh anggota kelompok yang mengembangkan Alternatif 2.)

Fungsi Kelayakan

(Diisi oleh anggota kelompok yang mengembangkan Alternatif 2.)

Fungsi Solusi

(Diisi oleh anggota kelompok yang mengembangkan Alternatif 2.)

Fungsi Objektif

(Diisi oleh anggota kelompok yang mengembangkan Alternatif 2.)

3.2.3 Alternatif 3 — *(Dikembangkan oleh anggota kelompok lain)*

Tujuan dan Heuristik Umum
(Diisi oleh anggota kelompok yang mengembangkan Alternatif 3.)
Himpunan Kandidat
(Diisi oleh anggota kelompok yang mengembangkan Alternatif 3.)
Himpunan Solusi
(Diisi oleh anggota kelompok yang mengembangkan Alternatif 3.)
Fungsi Seleksi
(Diisi oleh anggota kelompok yang mengembangkan Alternatif 3.)
Fungsi Kelayakan
(Diisi oleh anggota kelompok yang mengembangkan Alternatif 3.)
Fungsi Solusi
(Diisi oleh anggota kelompok yang mengembangkan Alternatif 3.)
Fungsi Objektif
(Diisi oleh anggota kelompok yang mengembangkan Alternatif 3.)

3.3 Analisis Efisiensi dan Efektivitas

(Akan diisi setelah ketiga alternatif selesai dikembangkan.)

3.4 Strategi *Greedy* yang Dipilih

(Akan diisi setelah ketiga alternatif selesai dikembangkan.)